

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

образовательное учреждение высшего образования

«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №5.

“ веб-приложения на Spring Boot. Основы Spring Security”

Выполнил: студент группы БВТ2402

Скопа Михаил Вячеславович

Москва, 2026

Цель работы: Изучить основы обеспечения безопасности в Spring Boot-приложении: научиться настраивать аутентификацию и авторизацию пользователей, ограничивать доступ к ресурсам приложения, работать с ролями и механизмами защиты запросов, а также познакомиться с основными подходами к хранению состояния пользователя в защищенной системе.

Задание для выполнения лабораторной работы

1. Добавьте отдельный endpoint регистрации администратора.

В файле AuthController.java создал эндпоинт для регистрации админа

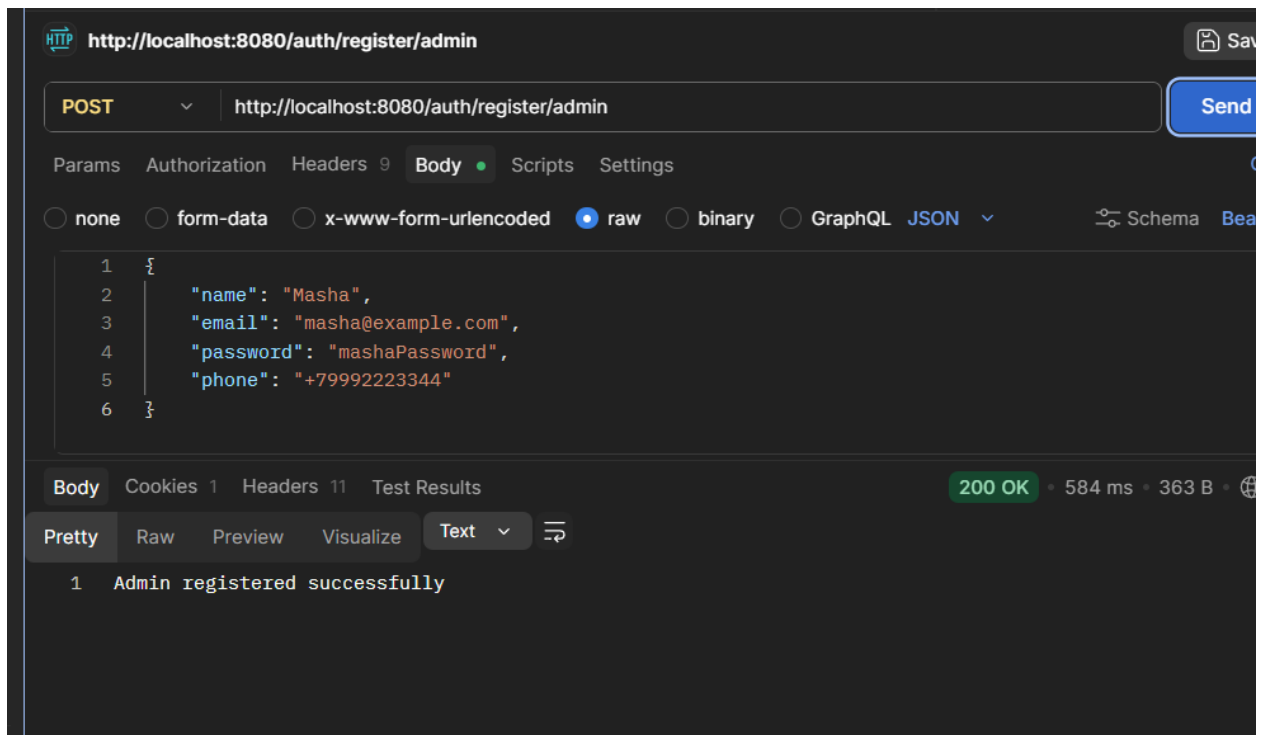
```
@PostMapping("/register/admin")
public ResponseEntity<?> registerAdmin(@RequestBody RegisterRequest request) {
    authService.registerAdmin(request);
    return ResponseEntity.ok(body: "Admin registered successfully");
}
```

В файле AuthService.java

```
public void registerAdmin(RegisterRequest request) { 1 usage
    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    user.setPhone(request.getPhone());

    user.setRole(UserRole.ROLE_ADMIN);

    userRepository.save(user);
}
```



`ROLE_ADMIN`

Создается пользователь с ролью Admin.

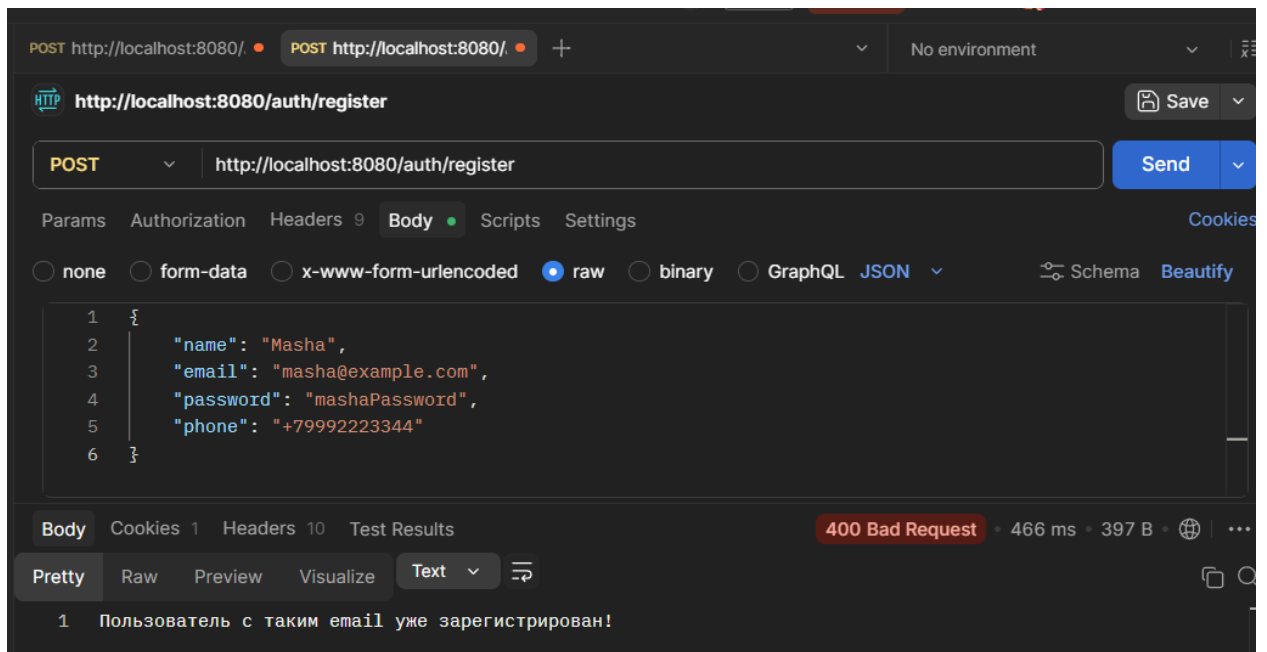
2. Реализуйте проверку уникальности email при регистрации.

Добавил проверку в AuthService:

```
public void register(RegisterRequest request) 1 usage
{
    if (userRepository.existsByEmail(request.getEmail())) {
        throw new IllegalArgumentException("Пользователь с таким email уже зарегистрирован!");
    }
    User user = new User();
```

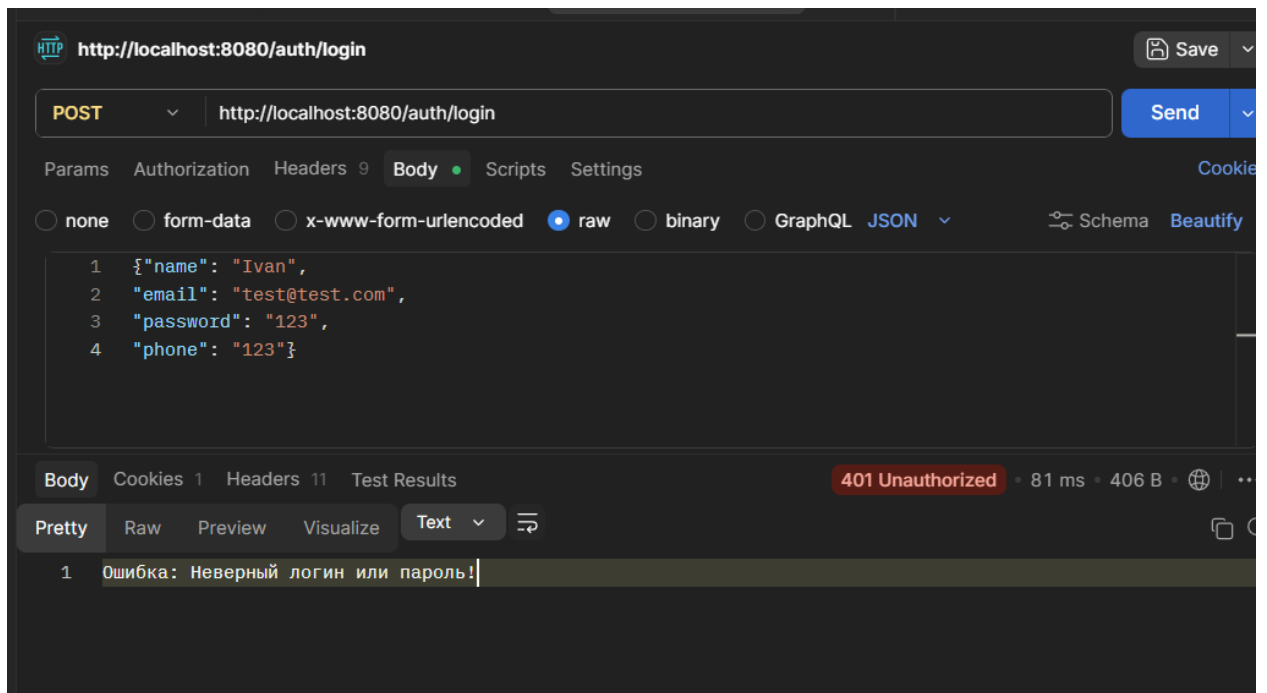
Обработал в AuthController.java

```
public class AuthController {private final AuthService authService;
    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody RegisterRequest request) {
        try {
            authService.register(request);
            return ResponseEntity.ok( body: "User registered successfully");
        } catch (IllegalArgumentException e) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
        }
    }
}
```



3. Добавьте обработку ошибки при неверном логине и пароле
В `JwtAuthController.java` изменил метод, добавив обработку ошибки:

```
@PostMapping("/login")
public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    try {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
        );
        String token = jwtService.generateToken(authentication.getName());
        return ResponseEntity.ok(token);
    } catch (org.springframework.security.authentication.BadCredentialsException e) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body("Ошибка: Неверный логин или пароль!");
    }
}
```



4. Вынесите преобразование сущности User в DTO в отдельный mapper.
Создал Mapper

```
package org.example.mapper;

import org.example.model.dto.UserDto;
import org.example.model.entity.User;
import org.springframework.stereotype.Component;

@Component
public class UserMapper {
    public UserDto toDto(User user) { 1 usage
        UserDto dto = new UserDto();
        dto.setName(user.getName());
        dto.setEmail(user.getEmail());
        dto.setPhone(user.getPhone());
        return dto;
    }
}
```

Добавил его в AuthService.java

```

public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final UserMapper userMapper;

    public UserDto getUserProfile(String email) { no usages
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new RuntimeException("User not found"));

        return userMapper.toDto(user);
    }
}

```

B AuthController:

```

@RestController
@RequestMapping("/test")
public class TestController {

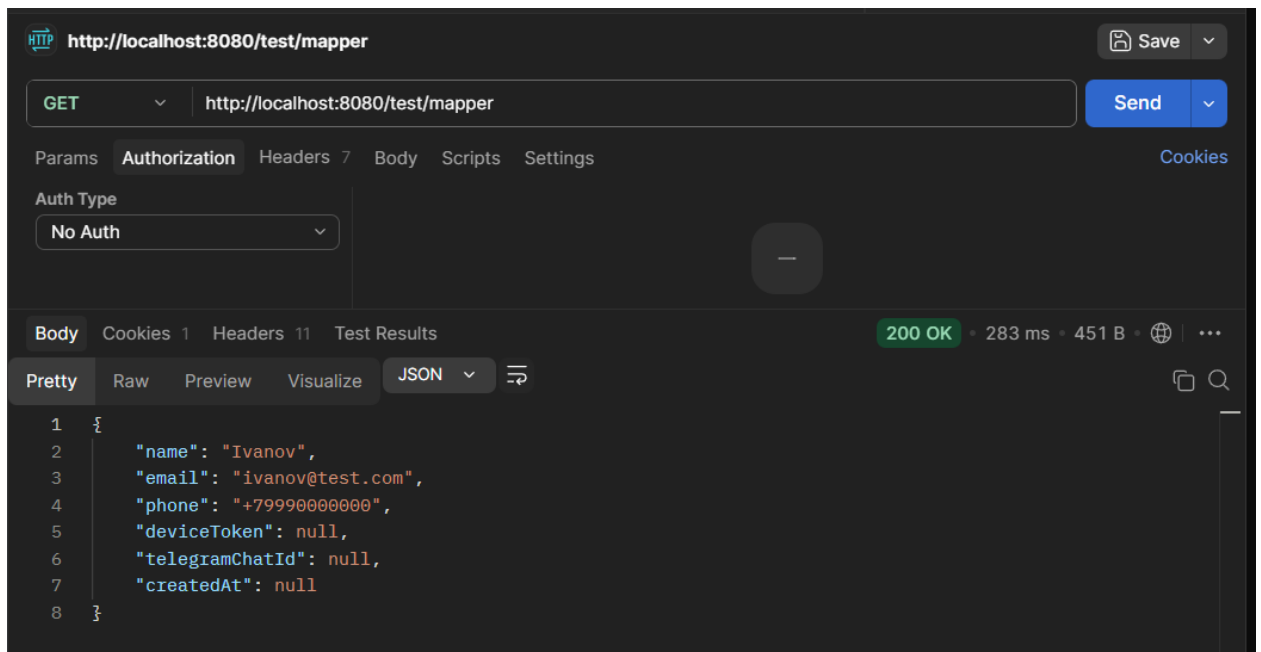
    private final UserMapper userMapper; 2 usages

    public TestController(UserMapper userMapper) { no usages
        this.userMapper = userMapper;
    }

    @GetMapping("/mapper")
    public UserDto checkMapper() {
        // Создаем фейкового юзера прямо в памяти
        User user = new User();
        user.setName("Ivanov");
        user.setEmail("ivanov@test.com");
        user.setPhone("+79990000000");

        // Вызываем маппер и возвращаем результат
        return userMapper.toDto(user);
    }
}

```



5. Реализуйте метод `extractUsername()` в `JwtService`

```
@Bean
public AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider(customUserDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder());
    return authProvider;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
    return configuration.getAuthenticationManager();
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests((AuthorizationManagerRequestMatcher<?> auth -> auth
            .requestMatchers(...patterns: "/auth/**").permitAll()
            .requestMatchers(...patterns: "/users/me").hasAuthority("ROLE_USER")
            .anyRequest().authenticated()
        )
        .sessionManagement((SessionManagementConfigurer<HttpSecurity> sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService; 2 usages
    private final CustomUserDetailsService userDetailsService; 2 usages

    public JwtAuthenticationFilter(JwtService jwtService, CustomUserDetailsService userDetailsService) { no usages
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain filterChain) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(7);
        final String username = jwtService.extractUsername(jwt);

        if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);

            UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());

            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
        filterChain.doFilter(request, response);
    }

    @Override 2 usages
    protected boolean shouldNotFilter(HttpServletRequest request) {
        return request.getServletPath().startsWith("/auth/");
    }
}

```



```

        .compact(),
    }

    @Value("${token.signing.key}")
    private String jwtSigningKey;

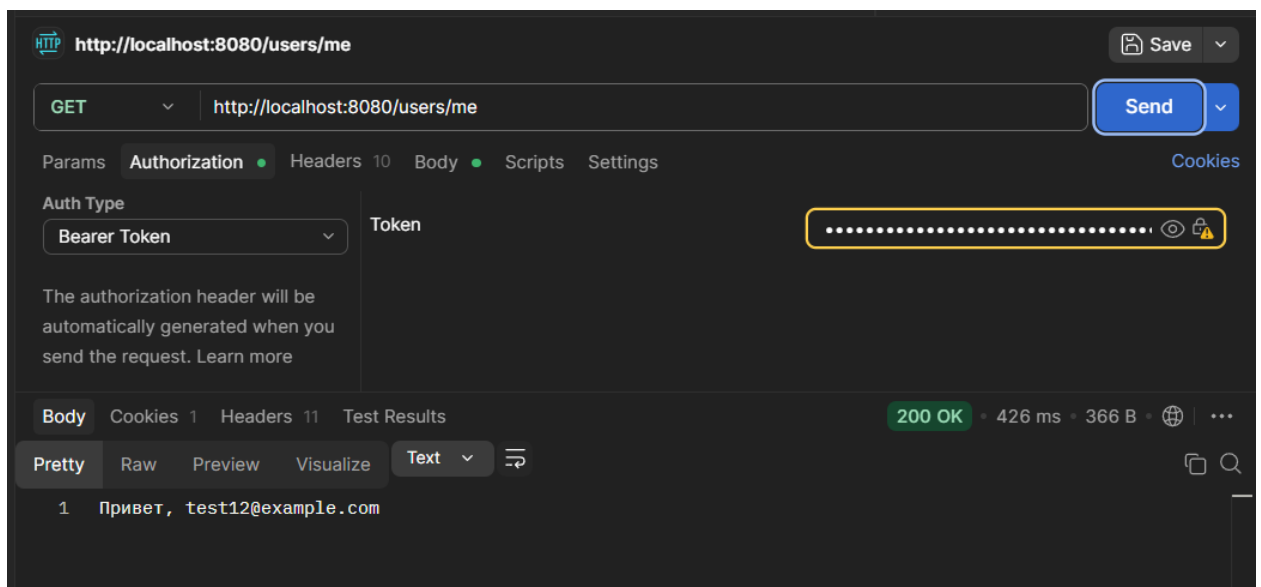
    public String extractUsername(String token) { 2 usages
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) { 2 usages
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) { 1 usage
        return Jwts.parser() JwtParserBuilder
            .verifyWith(getSignInKey())
            .build() JwtParser
            .parseSignedClaims(token) Jws<Claims>
            .getPayload();
    }

    private SecretKey getSignInKey() { 2 usages
        byte[] keyBytes = Decoders.BASE64.decode(jwtSigningKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }

```



6. Добавьте проверку срока действия токена.
Добавил проверку на время жизни токена:

```

if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
    UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);
    if (jwtService.isValid(jwt, userDetails)) {
        UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
            userDetails, credentials: null, userDetails.getAuthorities());
        authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}

filterChain.doFilter(request, response);

```

The screenshot shows a Postman interface for a GET request to `http://localhost:8080/users/me`. The 'Authorization' tab is selected, showing 'Bearer Token' as the auth type and a token field filled with dots. The 'Body' tab is also visible, showing the response: `Привет, test12@example.com`. The status bar at the bottom indicates a **200 OK** response with a 78 ms latency and 366 B of data.

The screenshot shows the same Postman interface, but the status bar at the bottom now indicates a **403 Forbidden** response with a 23 ms latency and 300 B of data. The 'Body' tab is selected, but the response content is not visible, only the number '1' is shown in the response list.

Токен срабатывает только пока действительный, после отведенного времени сервер выдает ошибку 403.

7. Настройте `SessionCreationPolicy.STATELESS` и полностью переключите приложение на JWT.

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .sessionManagement((SessionManagementConfigurer<HttpSecurity> sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(AbstractHttpConfigurer::disable)

        .authorizeHttpRequests((AuthorizationManagerRequestMatcher<SecurityContext> auth -> auth
            .requestMatchers(...patterns: "/auth/**").permitAll()
            .requestMatchers(...patterns: "/users/me").hasAuthority("ROLE_USER")
            .anyRequest().authenticated()
        ))

        .sessionManagement((SessionManagementConfigurer<HttpSecurity> sess -> sess.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```

POST http://loc... POST http://loc... POST http://loc... GET http://loc... GET http://loc...

http://localhost:8080/users/me

GET http://localhost:8080/users/me

Headers 10

Key	Value	Description	Bulk Edit	Presets	...
Authorization					
Cookie	JSESSIONID=FEA9166E3024905241E16...				
Postman-Token	<calculated when request is sent>				

Body Cookies 1 Headers 11 Test Results

200 OK • 102 ms • 366 B •

1 Привет, test12@example.com

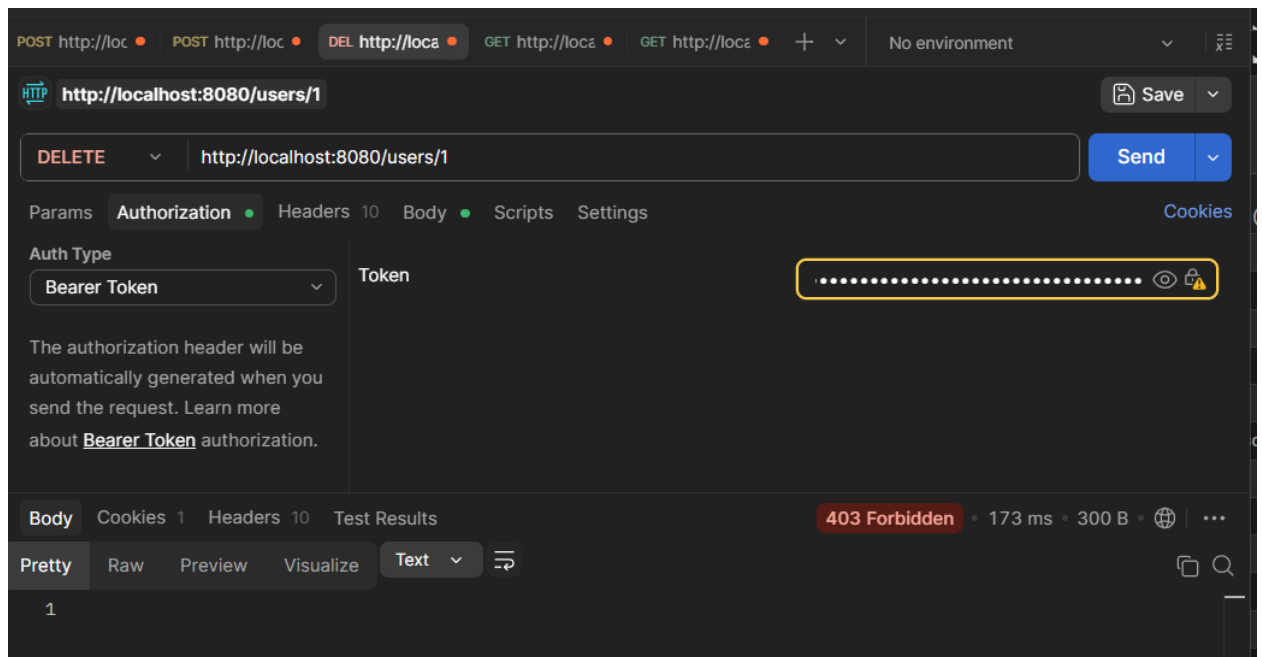
8. Защитите метод удаления пользователя через
`@PreAuthorize("hasRole('ADMIN')")`.

```

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public String deleteUser(@PathVariable Long id) {
    userService.deleteUser(id);
    return String.format("Пользователь %s удален", id);
}

```

Пользователь без прав администратора не может удалить другого пользователя



Вывод: изучил основы обеспечения безопасности в Spring Boot-приложении: научился настраивать аутентификацию и авторизацию пользователей, ограничивать доступ к ресурсам приложения, работать с ролями и механизмами защиты запросов, а также познакомился с основными подходами к хранению состояния пользователя в защищенной системе.